MINI INVENTORY

Component 2: Automated Deployment

Ansible managed service, Docker packaging and Kubernetes scaling with health probes

Student	Emaad Mirza
USN	1ms24cy020
Execution date	3 November 2026
Platform	Windows and Docker Desktop
Repository	https://github.com/EmaadMirza/Inventory-devops

This report records observed execution results. Configuration listings are the reviewed assignment source. Screenshots originate from actual terminal or Jenkins sessions. No coordinator invitation is claimed.

Abstract

The tested inventory JAR was deployed to an isolated SSH target using an Ansible role, variable, template and restart handler. A repeated run reported zero changes. A non-root Docker image passed live HTTP checks. The same image was loaded into a dedicated Kind cluster, exposed through a Kubernetes Service and scaled from three to five and back to three Ready replicas. Startup, readiness and liveness probes provide explicit health behavior.

Introduction

This component turns a built artifact into a managed application and a repeatable container deployment. Ansible expresses target configuration, Docker packages the runtime, and Kubernetes maintains the requested number of healthy instances. The demonstration keeps persistent inventory consistency outside scope: each replica owns an independent in-memory inventory.

Implementation: environment and sequence

Stages ran sequentially on Windows with existing Docker Desktop. Jenkins was stopped before the Ansible deployment. The Ansible target was stopped before cluster creation. A dedicated cluster name and .lab/kubeconfig isolated this work from existing Kubernetes contexts. Kubectl 1.36.1 was already supplied by Docker Desktop. Only Kind v0.33.0 was downloaded locally, and its SHA256 was checked against the official release. The pinned official node image is Kubernetes v1.36.4, matching kubectl's minor version.

Ansible inventory, role and variable

The inventory defines inventory-target in inventory_servers, reached over the private Compose network with a dedicated ED25519 key. SSH is not published to the Windows host, and password login is disabled. ansible-galaxy role init inventory_app actually generated a skeleton in .lab; the reviewed prepared role was reconciled into it. The role's default app_port is 8080, explicitly shown as a playbook variable. Defaults also supply the application directory and artifact source.

Tasks install the Java runtime and Supervisor, create /opt/mini-inventory, copy the JAR and render inventory.conf.j2. Changed files notify Restart inventory service; flush_handlers applies configuration before process and HTTP checks. Process readiness is retried to avoid racing Supervisor's STARTING state. The controller copies the mounted private key into a 0600 Linux file because Windows bind-mount permissions are unsuitable for OpenSSH private keys.

Managed service and platform adaptation

The isolated Debian target does not boot systemd. Supervisor is therefore the actual service manager, with autostart and autorestart enabled for the Java process. This is an explicit adaptation, not evidence of a systemd service run. The first play reported ok=8, changed=5, unreachable=0, failed=0 and executed the restart handler. The second reported ok=7, changed=0, unreachable=0, failed=0. The service kept PID 1243 across verification. A third actual run was captured on screen and again showed zero changes and HTTP status UP.

Docker image and API verification

The runtime image uses eclipse-temurin:17-jre-jammy, copies the executable JAR and runs as UID/GID 10001 with a 96 MiB maximum heap. The corrected Docker ignore rules permit the Ansible target entrypoint while excluding secrets and report/evidence directories from the application context. The image mini-inventory:1.0.0 built successfully. A 192 MiB test container bound only localhost:8083 and passed nine live HTTP checks before being stopped.

Kubernetes Deployment, Service and scaling

Kind loaded the already-built local image. The Deployment specifies three replicas and imagePullPolicy Never, avoiding a registry upload and repeated pulls. Its ClusterIP Service maps port 80 to container port 8080 and selects app=mini-inventory. The application uses a non-root security context, drops Linux capabilities and has a read-only root filesystem. Each replica requests 25 millicores/64 MiB and is limited to 250 millicores/160 MiB.

The original streaming image import stalled. A single-platform linux/amd64 archive was exported, copied into the dedicated node's root filesystem and successfully imported with containerd's supported --local mode. Docker's host settings and permissions were not changed. The reproducible procedure is in scripts/verify-kubernetes.ps1; initial unsuccessful transcripts are retained separately.

The deployment rolled out at three replicas, scaled to five Ready pods and returned to three. Each stage was verified against live pod readiness and captured in a terminal screenshot. Service endpoints and API responses were checked through localhost port-forwarding. Scaling evidence uses health and independent instances; it does not imply shared product data or load-test performance.

Enhancement rationale: health-aware deployment

A running process is insufficient evidence that an application can serve requests. The startup probe allows up to 60 seconds for initialization before other health checks begin. Readiness probes remove an unready instance from Service endpoints, while liveness probes permit Kubernetes to restart a persistently unhealthy container. All three call /health. The saved deployment and Ready pods verify the configured probes and healthy operation; no fault-injection or measured outage-recovery claim is made.

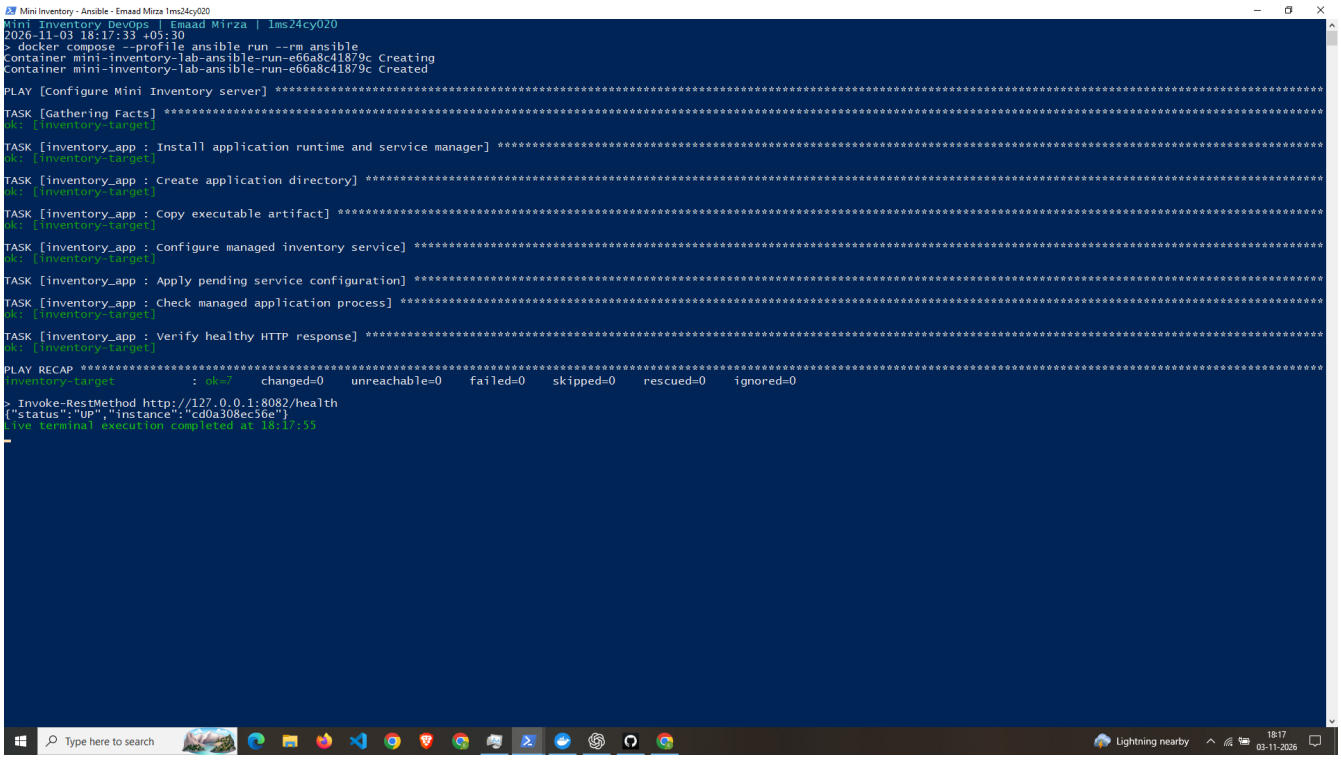
During the observed initial startup, probe events recorded connection refusals and timeouts before the pods became Ready. The three initial containers subsequently reached readiness without restarts. These transient events are retained rather than omitted; they demonstrate that startup readiness was actually evaluated, without constituting an induced failure-recovery experiment.

The node was later deliberately stopped to recover RAM and restarted to retake screenshots that had captured the wrong foreground window. Later screenshots therefore show restart counters from that node restart. Only screenshots inspected as the actual verification console are included; rejected captures remain outside the deliverables in ignored .lab.

Actual results

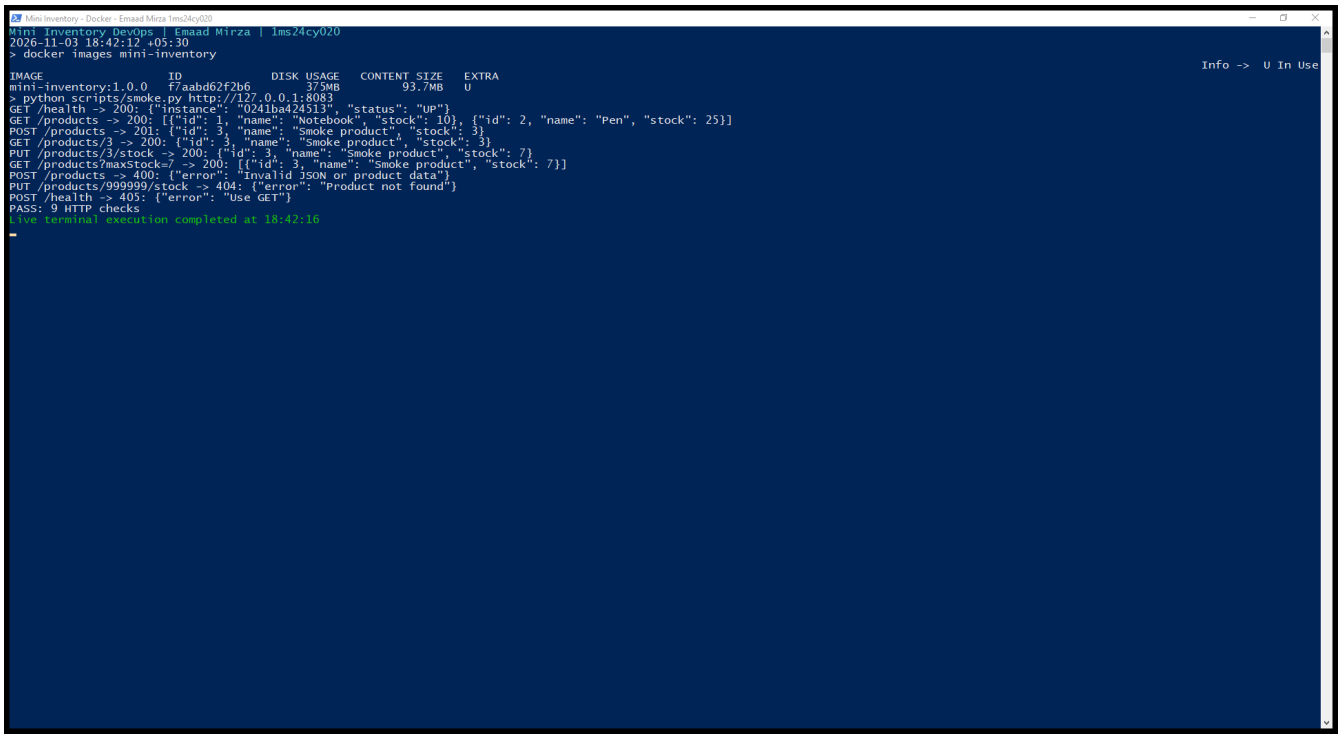
Check	Actual result
Role generator	ansible-galaxy role init succeeded
Ansible first run	ok=8 changed=5 unreachable=0 failed=0; handler executed
Ansible second run	ok=7 changed=0 unreachable=0 failed=0
Managed service	Supervisor RUNNING; /health returned UP
Docker image	Build succeeded; 9 live HTTP checks passed
Kubernetes scaling	3 Ready -> 5 Ready -> 3 Ready
Service and health	Live endpoints and HTTP responses saved in evidence
Health probes	Startup/readiness/liveness configured; final pods Ready

Evidence: idempotent Ansible run and managed health



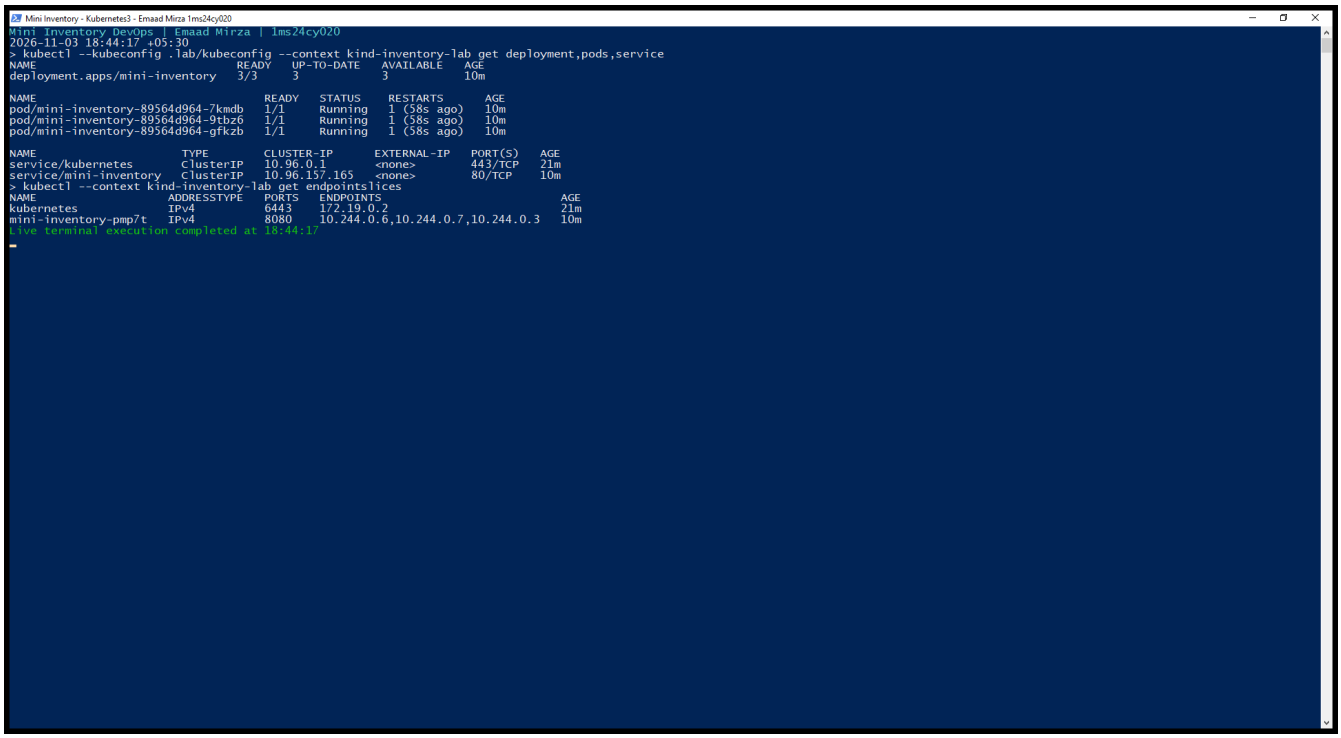
Original screenshot: evidence/terminal-Ansible.png. Embedded without alteration; zoom to inspect command output.

Evidence: live Docker API checks



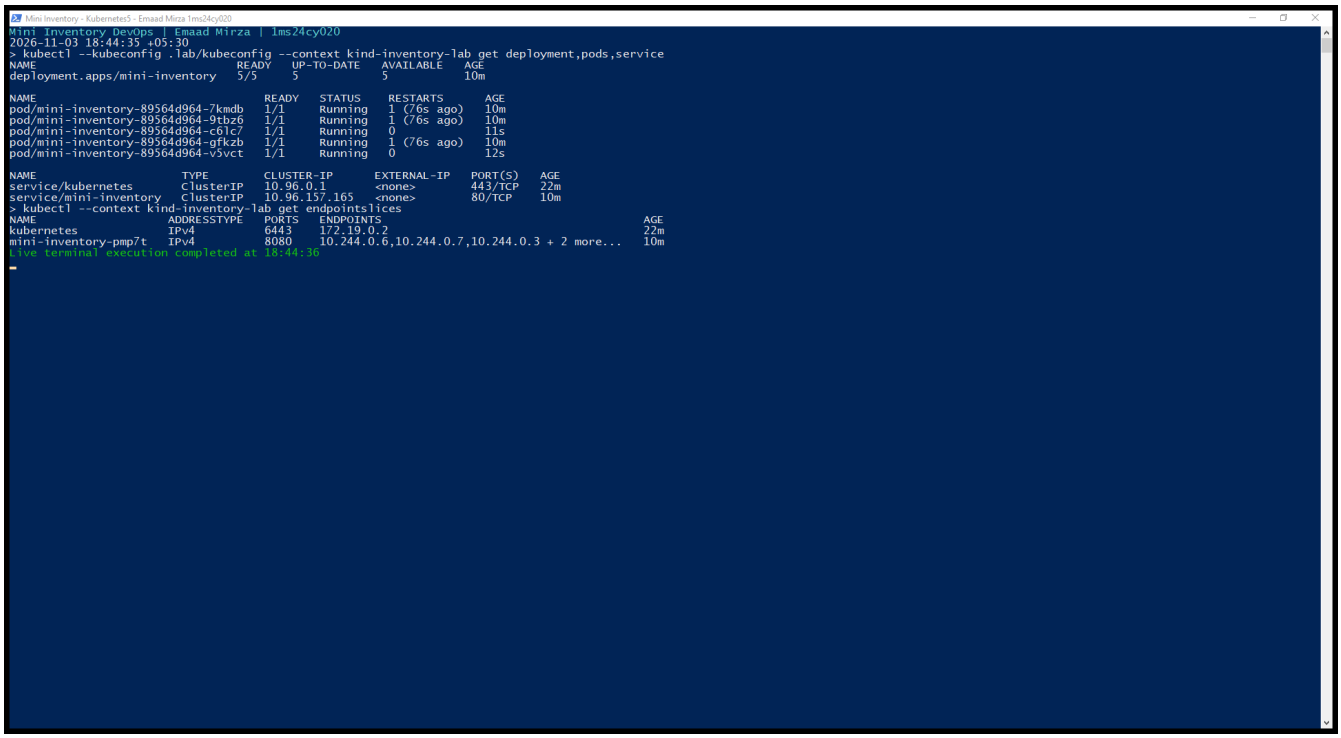
Original screenshot: evidence/terminal-Docker.png. Embedded without alteration; zoom to inspect command output.

Evidence: initial three Ready replicas



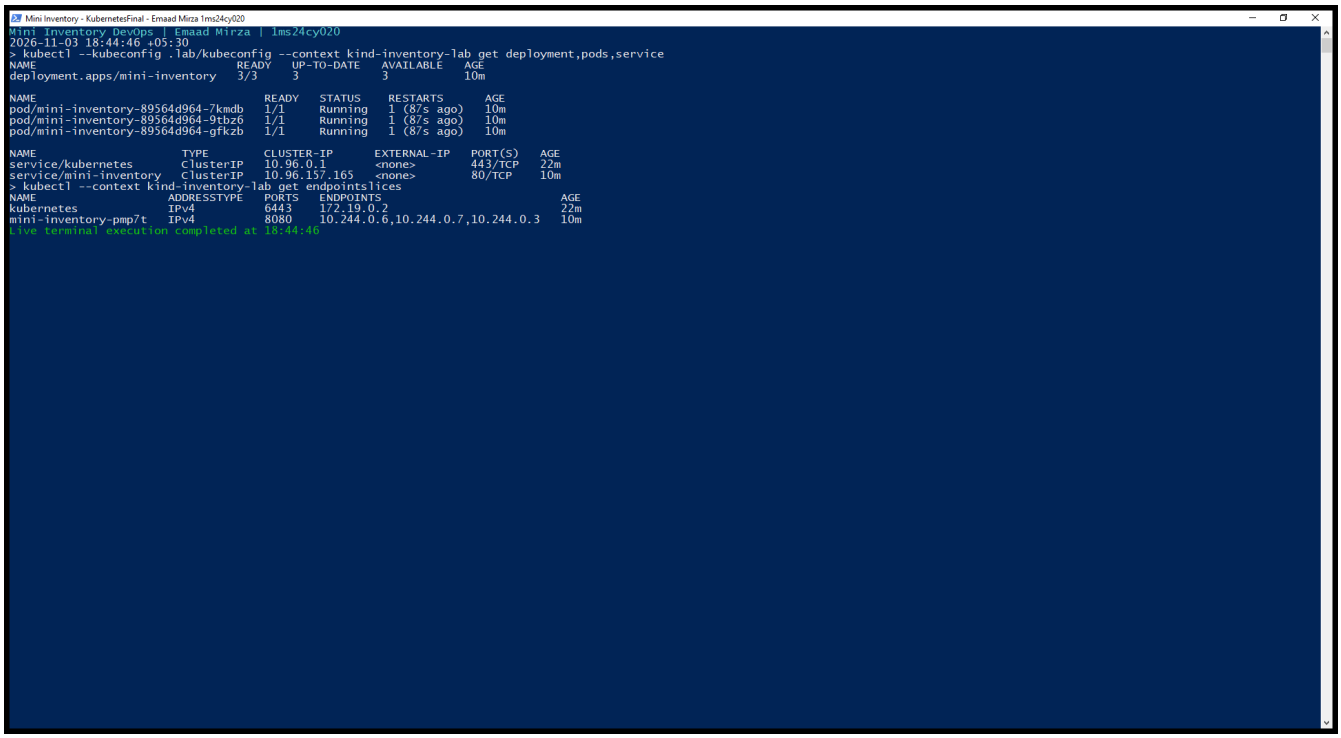
Original screenshot: evidence/terminal-Kubernetes3.png. Embedded without alteration; zoom to inspect command output.

Evidence: five Ready replicas



Original screenshot: evidence/terminal-Kubernetes5.png. Embedded without alteration; zoom to inspect command output.

Evidence: return to three Ready replicas



Original screenshot: evidence/terminal-KubernetesFinal.png. Embedded without alteration; zoom to inspect command output.

Conclusion

The deployment chain was executed using a managed SSH target, an idempotent Ansible role, a verified Docker image and a dedicated Kubernetes cluster. The requested scaling cycle completed with health probes present and replicas Ready. The main limitation is process-local inventory. Shared persistence, application authentication and production-grade multi-node infrastructure remain future work. Neither a production high-availability guarantee nor fault-recovery performance is claimed.

References

Ansible handlers. https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_handlers.html. Accessed 3 November 2026.

Ansible roles. https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html. Accessed 3 November 2026.

Dockerfile reference. https://docs.docker.com/reference/dockerfile/. Accessed 3 November 2026.

Kind v0.33.0 release and node digests. https://github.com/kubernetes-sigs/kind/releases/tag/v0.33.0. Accessed 3 November 2026.

Kind image-store known issues. https://kind.sigs.k8s.io/docs/user/known-issues/. Accessed 3 November 2026.

Kubernetes health probes. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/. Accessed 3 November 2026.

Kubernetes Deployments. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/. Accessed 3 November 2026.

Code and configuration listings

compose.yaml

```
name: mini-inventory-lab
services:
  jenkins:
    build: ./infra/jenkins
    ports: ["127.0.0.1:18081:8080"]
    environment:
      JAVA_OPTS: -Xms128m -Xmx512m -Djenkins.install.runSetupWizard=false
    mem_limit: 1g
    volumes:
      - jenkins-data:/var/jenkins_home
      - ./infra/jenkins/bootstrap.groovy:/var/jenkins_home/init.groovy.d/bootstrap.groovy:ro
      - ./.secrets/jenkins-admin-password:/run/secrets/jenkins-admin-password:ro
    profiles: [ci]
  inventory-target:
    build:
      context: .
      dockerfile: infra/ansible-target/Dockerfile
    volumes: ["./.secrets:/keys:ro"]
    ports: ["127.0.0.1:8082:8080"]
    mem_limit: 384m
    profiles: [ansible]
  ansible:
    build: ./infra/ansible-controller
    volumes: ["./:/workspace:ro", "./.secrets:/keys:ro"]
    mem_limit: 384m
    profiles: [ansible]
    command: ["-i", "inventory.ini", "playbook.yml"]
  maven:
    image: maven:3.9-eclipse-temurin-17
    working_dir: /workspace
    volumes: ["./:/workspace", "maven-data:/root/.m2"]
    environment:
      MAVEN_OPTS: -Xmx256m
    mem_limit: 512m
    profiles: [build]
    command: ["mvn", "-B", "-ntp", "clean", "verify"]
volumes:
  jenkins-data:
  maven-data:
```

ansible/inventory.ini

```
[inventory_servers]
inventory-target ansible_user=root ansible_ssh_private_key_file=/tmp/inventory-key

[all:vars]
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=accept-new -o UserKnownHostsFile=/tmp/known_hosts'
```

ansible/playbook.yml

```
---
- name: Configure Mini Inventory server
  hosts: inventory_servers
  vars:
    app_port: 8080
  roles:
    - inventory_app
```

ansible/roles/inventory_app/defaults/main.yml

```
---
app_port: 8080
app_directory: /opt/mini-inventory
artifact_source: /workspace/target/mini-inventory.jar
```

ansible/roles/inventory_app/tasks/main.yml

```
---
- name: Install application runtime and service manager
  ansible.builtin.apt:
    name: [openjdk-17-jre-headless, supervisor]
    state: present
    update_cache: true
    cache_valid_time: 3600
    install_recommends: false

- name: Create application directory
  ansible.builtin.file:
    path: "{{ app_directory }}"
    state: directory
    mode: '0755'

- name: Copy executable artifact
  ansible.builtin.copy:
    src: "{{ artifact_source }}"
    dest: "{{ app_directory }}/app.jar"
    mode: '0644'
  notify: Restart inventory service

- name: Configure managed inventory service
  ansible.builtin.template:
    src: inventory.conf.j2
    dest: /etc/supervisor/conf.d/inventory.conf
    mode: '0644'
  notify: Restart inventory service

- name: Apply pending service configuration
  ansible.builtin.meta: flush_handlers

- name: Check managed application process
  ansible.builtin.command: supervisorctl status inventory
  register: inventory_service
  changed_when: false
  failed_when: "'RUNNING' not in inventory_service.stdout"
  until: "'RUNNING' in inventory_service.stdout"
  retries: 10
  delay: 1

- name: Verify healthy HTTP response
  ansible.builtin.uri:
    url: "http://127.0.0.1:{{ app_port }}/health"
    status_code: 200
  register: health_check
  until: health_check.status == 200
  retries: 20
  delay: 2
```

ansible/roles/inventory_app/handlers/main.yml

```
---
- name: Restart inventory service
  ansible.builtin.shell: |
    set -eu
    supervisorctl reread
    supervisorctl update
    supervisorctl restart inventory
  changed_when: true
```

ansible/roles/inventory_app/templates/inventory.conf.j2

```
[program:inventory]
command=/usr/bin/java -Xms16m -Xmx96m -jar {{ app_directory }}/app.jar
directory={{ app_directory }}
environment=APP_PORT="{{ app_port }}"
autostart=true
autorestart=true
startsecs=2
stdout_logfile=/var/log/inventory.out.log
stderr_logfile=/var/log/inventory.err.log
```

infra/ansible-controller/Dockerfile

```
FROM python:3.12-slim-bookworm
RUN apt-get update && apt-get install -y --no-install-recommends openssh-client && rm -rf
    /var/lib/apt/lists/*
RUN pip install --no-cache-dir ansible-core==2.18.6
WORKDIR /workspace/ansible
COPY entrypoint.sh /entrypoint.sh
RUN chmod 755 /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

infra/ansible-controller/entrypoint.sh

```
#!/bin/sh
set -eu
cp /keys/id_ed25519 /tmp/inventory-key
chmod 600 /tmp/inventory-key
exec ansible-playbook "$@"
```

infra/ansible-target/Dockerfile

```
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends openssh-server python3 supervisor ca-
    certificates && rm -rf /var/lib/apt/lists/* && mkdir -p /run/sshd /root/.ssh && chmod 700 /root/.ssh
COPY infra/ansible-target/start.sh /start.sh
RUN chmod 755 /start.sh
ENTRYPOINT ["/start.sh"]
```

infra/ansible-target/start.sh

```
#!/bin/sh
set -eu
cp /keys/id_ed25519.pub /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
ssh-keygen -A
supervisord -c /etc/supervisor/supervisord.conf
exec /usr/sbin/sshd -D -e -o PasswordAuthentication=no -o PermitRootLogin=prohibit-password
```

Dockerfile

```
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY target/mini-inventory.jar app.jar
USER 10001:10001
ENV APP_PORT=8080
EXPOSE 8080
ENTRYPOINT ["java","-Xms16m","-Xmx96m","-jar","/app/app.jar"]
```

k8s/kind.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
```

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mini-inventory
spec:
  replicas: 3
  selector:
    matchLabels: {app: mini-inventory}
  template:
    metadata:
```

```
      labels: {app: mini-inventory}
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
      containers:
        - name: api
          image: mini-inventory:1.0.0
          imagePullPolicy: Never
          ports:
            - containerPort: 8080
          env:
            - name: APP_PORT
              value: "8080"
          resources:
            requests: {cpu: 25m, memory: 64Mi}
            limits: {cpu: 250m, memory: 160Mi}
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: [ALL]}
          startupProbe:
            httpGet: {path: /health, port: 8080}
            periodSeconds: 2
            failureThreshold: 30
          readinessProbe:
            httpGet: {path: /health, port: 8080}
            periodSeconds: 5
          livenessProbe:
            httpGet: {path: /health, port: 8080}
            periodSeconds: 10
```

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mini-inventory
spec:
  type: ClusterIP
  selector: {app: mini-inventory}
  ports:
    - port: 80
      targetPort: 8080
```